

RVC Backstage — Task Management System Plan

RVC Backstage — Task Management System Plan

Task management system for **Raw Vision Collective (RVC)** — an international-scale concert promoter / event organizer (rawvision.demo-wit.id).

Working name: **RVC Backstage** (“everything behind the show”). System UI language: **English**.

Indonesian version: PLAN.id.md

1. Vision & goals

- One workspace per event (concert), shared across all divisions, with clear ownership per task.
- Fast to adopt: a staff member should understand “My Tasks → do → done” in under 5 minutes.
- Cross-division by design: handoffs and dependencies between divisions are first-class.
- External collaborators (vendors, artist management, venue, sponsors) can submit and update inside the system — scoped, safe, auditable.
- The Owner gets an executive summary: portfolio health, budget burn, pending approvals with one-click approve/reject.

2. Design language

Derived from the RVC website (rawvision.demo-wit.id):

- Monochrome palette: black / white / gray, high contrast. Neutral backdrop; event poster imagery provides the color.
- Modern geometric sans-serif, typography-driven hierarchy, generous whitespace, gallery-like grid.
- Signature elements to reuse: **countdown timer to show day** on every event workspace, up-right arrow (↗) motifs for links/actions, minimalist logo lockup.
- Dark theme as default (backstage vibe), light theme available. Flat surfaces, hairline borders, no decorative gradients.

3. Organizational structure

Executive - Owner / CEO — final approvals, executive dashboard, full visibility. - (Optional later: COO / Managing Director with delegated approval rights.)

Divisions (each has one Division Head + Staff):

#	Division	Responsibilities (examples)
1	Talent & Booking	Artist scouting, offers, booking contracts, rider intake, artist advancing
2	Production	Stage, sound, lighting, video/visuals, SFX, technical rider fulfillment, site build, soundcheck

#	Division	Responsibilities (examples)
3	Operations & Logistics	Venue coordination, scheduling, transport, freight/customs for international acts, accommodation, accreditation, catering
4	Security & Safety	Crowd management plan, security vendor, medical, emergency response
5	Hospitality & Artist Liaison	Artist handling, backstage/green rooms, hospitality riders, VIP guests
6	Marketing & Communications	Brand, creative assets, content calendar, social media, PR/media partners, announcements
7	Ticketing & Sales	Ticketing platform setup, pricing tiers, presales, box office, gate ops, daily sales reporting
8	Sponsorship & Partnership	Sponsor pipeline, proposals, activation delivery, post-event reports
9	Finance	Event budgets, expense/PO processing, vendor payments, artist fees & tax, settlement
10	Legal & Licensing	Permits (police/city/venue), performance licenses, immigration/work permits, contract review, insurance
11	HR & Volunteers	Crew recruitment, volunteer management, shift scheduling, briefings

External collaborators (guest access, event-scoped): - Vendors & suppliers (sound system, staging, catering, merch...) - Artist management / booking agencies - Venue representatives - Sponsor representatives - Freelance crew

4. Roles & permissions

Five roles. Everything is enforced in the API/service layer (single authorization module), not just in the UI — optionally hardened later with native Postgres RLS.

Capability	Owner	Admin	Division Head	Staff	External
View all events & all divisions	✓	✓	summary only	—	—
View own division tasks (all events)	✓	✓	✓	✓	—
View assigned tasks only	✓	✓	✓	✓	✓ (scoped to invite)
Create / archive events	✓	✓	—	—	—
Manage users, divisions, settings	✓	✓	—	—	—
Create / edit tasks in own division	✓	✓	✓	✓	—

Capability	Owner	Admin	Division Head	Staff	External
Assign tasks within division	✓	✓	✓	self/peers	—
Cross-division handoff request	✓	✓	✓	✓	—
Update status / comment / upload on assigned tasks	✓	✓	✓	✓	✓
Submit structured forms (quotes, riders, manifests)	—	—	—	—	✓
Invite external collaborators (own division, per event)	✓	✓	✓	—	—
Review/accept external submissions	✓	✓	✓	assigned staff	—
Create budget / expense requests	✓	✓	✓	✓ (needs Head approval)	—
View budgets	all	all	own division	—	—
Approve: division-level (tier 1)	✓	—	✓	—	—
Approve: final / high-value / contracts / artist offers	✓	—	—	—	—
Executive dashboard	✓	read-only (optional)	—	—	—
Audit log	✓	✓	—	—	—

Principles: - **Division-scoped by default** — staff see their division's world plus anything they're assigned to or watching. - **Event-scoped for externals** — a guest invited to "YE Live Concert / Production" sees only their own tasks and forms in that event. Never budgets, never other vendors, never internal tasks. - **Financial data** — Owner, Admin, Finance division see all; a Division Head sees only their own division's budget lines.

5. External collaborator model

- Invited by a Division Head via email → **magic-link login** (no password friction).
- Scope of invite: one event + one division + explicit task/form assignments.

- What they can do: view assigned tasks, update status, comment, upload deliverables, fill structured forms:
 - Vendor quotation form
 - Technical rider form (artist management)
 - Logistics manifest (freight, equipment lists)
 - Crew / guest lists
- Every external submission lands in a **review queue** for the owning division (accept / request changes).
- Invites expire (default: 7 days after event settlement); access is revocable at any time; all actions audited.

6. Core modules

6.1 Events (workspaces)

Each concert = one event workspace. Fields: name, artist(s), venue, show date, capacity, status, cover image (poster). Lifecycle phases: **Planning** → **Pre-production** → **Promotion** → **Show week** → **Show day** → **Settlement**. Health status per event (On track / At risk / Critical) auto-computed from overdue + blocked tasks and budget burn.

6.2 Event templates (“playbooks”)

Creating an event from the “International Concert” template auto-generates each division’s standard checklist (e.g. Legal: permits checklist with lead times; Production: technical advance checklist). Templates are editable; this is the biggest “fast to use” lever.

6.3 Tasks

Title, description, division, event, assignees, watchers, priority (Low/Medium/High/Urgent), start/due dates, status (**Backlog** → **To do** → **In progress** → **In review** → **Blocked** → **Done**), subtasks/checklist, labels, attachments, comments with @mentions, dependencies (blocked by / blocks), recurring tasks.

6.4 Cross-division handoffs

A task can request work from another division (“Marketing needs stage design render from Production”). The receiving Division Head accepts → it appears in their board, linked as a dependency. No more lost WhatsApp requests.

6.5 Views

- **My Tasks** (default landing for staff — today / this week / overdue)
- Kanban per division per event
- List with saved filters
- Timeline (Gantt) per event — critical path to show day
- Calendar (deadlines, milestones, show dates)

6.6 Run of show

Minute-by-minute rundown for show day (doors, opener, changeover, headliner, curfew), owned by Production/Ops, readable by all divisions, printable/exportable.

6.7 Files & documents

Attachments on tasks + a per-event document library (contracts, permits, riders, stage plots) with division-level access control.

6.8 Budget & procurement

Budget per event → budget lines per division → expense requests (amount, vendor, justification, quote attachment) flowing into the approval engine. Committed vs actual vs budget, visible to Finance and Owner.

6.9 Approvals engine

Generic multi-step approval chains, configurable per type:

Type	Chain (default)
Expense ≤ threshold A	Division Head
Expense threshold A–B	Division Head → Finance
Expense > threshold B	Division Head → Finance → Owner
Artist offer	Talent Head → Finance → Owner
Contract (any)	Legal review → Owner
Sponsorship deal	Sponsorship Head → Legal → Owner
Public content/announcement	Marketing Head

Thresholds and currency configurable (org settings). Each step: approve / reject / request changes, with comment. Full history retained.

6.10 Notifications

Trigger	In-app	Email	Recipient
Task assigned to you	✓	✓	assignee
@mention in comment	✓	✓	mentioned user
Due in 24h / overdue	✓	✓	assignee + Head (overdue)
Dependency resolved (unblocked)	✓	—	assignee
Approval requested	✓	✓	approver
Approval decided	✓	✓	requester
External submission received	✓	✓	division reviewers
Handoff request	✓	✓	receiving Head
Daily digest (opt-in)	—	✓	all internal
Weekly executive digest	—	✓	Owner

In-app = realtime via SSE (server-sent events). Later option: WhatsApp/Telegram bridge.

6.11 Executive dashboard (Owner)

- **Portfolio cards:** every active event with countdown, phase, health status, budget burn %.

- **Pending approvals queue:** approve/reject inline with comment — the Owner's main action surface.
- Budget vs actual per event; total committed across portfolio.
- Upcoming milestones (next 14 days) + cross-division blockers and overdue hotspots.
- Ticket sales snapshot (manual daily entry by Ticketing in MVP; integration later).
- Recent activity feed.

6.12 Audit & activity log

Every create/update/approval/permission change logged (who, what, when). Filterable; visible to Owner/Admin.

6.13 Search

Global search across tasks, events, files, people — scoped by permissions.

7. Tech stack

Self-hosted — no managed cloud services. Database is local PostgreSQL on the company server; the app deploys to the same server.

Layer	Choice	Why
App	Next.js (App Router, TypeScript) + Tailwind + shadcn/ui restyled to RVC monochrome	Full-stack in one codebase, matches the minimal aesthetic
Database	PostgreSQL (local, self-hosted) + Drizzle ORM + migrations	Full control, no vendor lock-in
Auth	Auth.js (NextAuth v5): email+password for internal staff, magic link for external guests	Guest access without password friction
Authorization	Central permission module in the service layer (role + division + event scoping)	One place to audit; optional Postgres RLS hardening later
Realtime notifications	SSE (server-sent events)	No extra infrastructure needed
File storage	Server disk (/up loads , nginx-served, auth-gated); MinIO later if needed	Simple, backed up with the server
Email	SMTP (company SMTP or Resend)	Transactional + digests
Background jobs	node-cron (due reminders, digests, health recompute)	Runs inside the app process
Deployment	Docker Compose (Next.js + Postgres + nginx reverse proxy) on the company server	Reproducible, easy backup/restore

8. Data model (main tables)

profiles, divisions, division_members (user + division + role), events, event_divisions, tasks, task_assignees, task_watchers, task_dependencies, task_checklist_items, labels, comments, attachments, documents, budgets, budget_lines, expense_requests, approvals, approval_steps, form_templates, form_submissions, external_invites, notifications, activity_log, run_of_show_items, ticket_sales_snapshots, event_templates.

9. Roadmap

Phase 1 — Foundation (usable from day one) Auth + org structure + roles (central authorization module), events, tasks (list + kanban), My Tasks, comments/@mentions, attachments, in-app notifications, RVC monochrome theme + event countdown.

Phase 2 — Owner layer Approvals engine, budgets + expense requests, executive dashboard, email notifications, activity log.

Phase 3 — External & scale Guest portal (magic link), structured submission forms + review queue, event templates/playbooks, timeline (Gantt) + calendar, run of show, ticket sales snapshots.

Phase 4 — Polish Daily/weekly digests, reports & exports (event settlement report), global search, audit UI, mobile/PWA polish.

10. Open questions

1. Approval thresholds & currency (IDR? USD? both?) — defaults proposed, needs Owner's numbers.
2. Notification channels beyond email — WhatsApp/Telegram needed at launch?
3. Ticketing: manual daily snapshot at MVP OK, or is there a ticketing platform API to integrate?
4. Single organization (RVC only) or multi-brand support later?